

Evaluación preliminar de calidad de agua con QSVM

Evaluación reproducible de tres mapas de características para el ODS 6

QuantumTenango — Quantathon CR 2026, Challenge 2

24 de julio de 2026

Resumen

El acceso al agua potable, en consonancia con el ODS 6, requiere herramientas de diagnóstico rápido para apoyar la toma de decisiones. En este trabajo abordamos la clasificación binaria de la potabilidad del agua ($\text{Potability} \in \{0, 1\}$) mediante una comparación entre una máquina de vectores de soporte (SVM) y su contraparte con kernel cuántico (QSVM). Aprovechamos la transformación del espacio de características inducida por los kernels cuánticos para explorar relaciones complejas entre mediciones fisicoquímicas de la calidad del agua. Mediante selección forward elegimos cinco variables; con ellas, la SVM-RBF alcanzó $F1 = 0.533$ y $\text{ROC-AUC} = 0.693$ sobre 656 observaciones reservadas. Después construimos kernels de fidelidad con los mapas ZZ, Pauli $Z + YY$ y $R_y\text{-CX-}R_x$, y los evaluamos en submuestras balanceadas de 16/8 y 32/16 observaciones de train/test. El mejor resultado cuántico correspondió a Pauli $Z + YY$ con 32 muestras de entrenamiento: $F1 = 0.625$ y $\text{ROC-AUC} = 0.766$ sobre 16 casos de test. Por el tamaño reducido y la ausencia de ejecución en una QPU, estos resultados demostraron una implementación funcional, no una ventaja cuántica.

1. Descripción del problema e impacto

En este proyecto nos propusimos predecir la potabilidad del agua (Potability) a partir del conjunto *Water Potability* [2], que contiene 3,276 observaciones, nueve variables y una proporción de 39.01 % de muestras potables. Interpretamos el modelo como una herramienta de preclasificación de agua para apoyar la toma de decisiones. Esto no sustituye un mecanismo más completo de pruebas de laboratorio, pero da un

Interpretamos el clasificador como un mecanismo de apoyo para una intervención más temprana. Sin embargo, no pretendimos que sustituyera las pruebas de laboratorio, especialmente ante el riesgo de clasificar como potable una muestra que no lo fuera.

Relacionamos el proyecto con el ODS 6, pues el clasificador podría apoyar una infraestructura de monitoreo asequible. Para un despliegue real serían necesarios sensores calibrados, validación local, umbrales sensibles al costo del error, conectividad y confirmación de laboratorio de la calidad del agua.

2. Preparación y línea base clásica

Realizamos una partición estratificada 80/20 con semilla 42: usamos 2,620 registros para train y reservamos 656 para test. Para tratar los valores faltantes de **ph**, **Sulfate** y **Trihalomethanes**, calculamos la mediana **exclusivamente con los datos de train** y la aplicamos a ambas particiones. Posteriormente estandarizamos las variables para entrenar la SVM-RBF. Para la etapa cuántica, reescalamos las cinco variables seleccionadas al intervalo angular empleado por los circuitos.

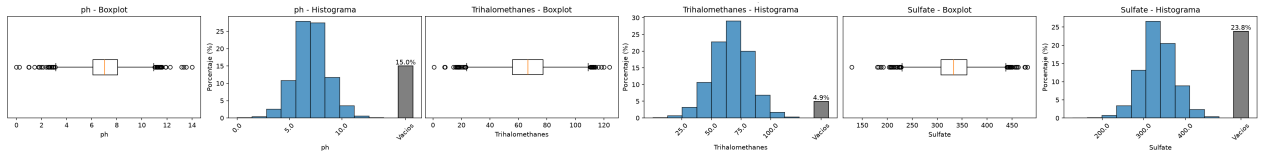


Figura 1: Distribución, valores atípicos y proporción de datos faltantes de las nueve variables. Observamos valores faltantes en **ph** (15.0 %), **Sulfate** (23.8 %) y **Trihalomethanes** (4.9 %), lo que motivó la imputación con medianas calculadas exclusivamente sobre train.

2.1. Selección secuencial hacia adelante

Como no todas las mediciones necesariamente aportaban el mismo valor discriminativo, aplicamos selección secuencial hacia adelante (*forward selection*) sobre train. En cada paso añadimos la variable que maximizó el F1 promedio de cinco folds estratificados; durante esta fase fijamos provisionalmente $C = 10$ y $\gamma = \text{auto}$. Mantuvimos los datos de test

Cuadro 1: Trayectoria de la selección forward en validación cruzada.

p	Variable incorporada	Conjunto elegido	F1	AUC	Recall
1	ph	ph	0.5140	0.5215	0.7104
2	Hardness	anterior + Hardness	0.5363	0.5615	0.7182
3	Chloramines	anterior + Chloramines	0.5103	0.5738	0.6192
4	Sulfate	anterior + Sulfate	0.5433	0.6879	0.5255
5	Solids	anterior + Solids	0.5839	0.7207	0.5694
6	Conductivity	anterior + Conductivity	0.5746	0.7147	0.5744
7	Trihalomethanes	anterior + Trihalomethanes	0.5795	0.7092	0.5851
8	Turbidity	anterior + Turbidity	0.5689	0.7087	0.5733
9	Organic_carbon	anterior + Organic_carbon	0.5630	0.6949	0.5812

fuera de todo el proceso de selección. En el Cuadro 1 mostramos cómo incorporamos las variables y cómo evolucionaron las métricas de validación.

Elegimos el subconjunto de cinco variables porque allí observamos el máximo simultáneo de F1 (0.5839) y AUC (0.7207). A partir de ese punto, añadir variables no produjo una mejora sostenida: el F1 bajó a 0.5746 con seis variables y permaneció por debajo del máximo en los pasos siguientes. Así justificamos el uso de cinco qubits, uno para cada característica seleccionada: ph, Hardness, Solids, Chloramines y Sulfate.

2.2. Ajuste final y línea base

Con las cinco variables seleccionadas evaluamos los hiperparámetros:

$$\begin{cases} C \in \{0.1, 1, 10\}, \\ \gamma \in \{\text{scale}, \text{auto}, 0.01\}, \end{cases}$$

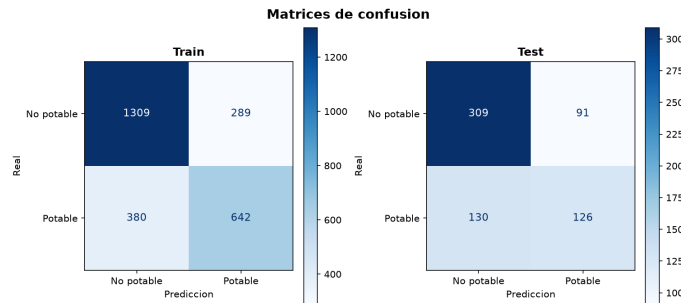
donde C fue el parámetro de regularización y γ determinó la influencia de cada muestra de entrenamiento sobre la frontera de decisión. Seleccionamos la combinación por el F1 promedio de cinco folds y obtuvimos como mejor configuración $C = 10, \gamma = \text{auto}$.

Cuadro 2: Línea base SVM–RBF sobre el conjunto completo.

Partición	Exactitud	Precisión	Recall	F1	ROC–AUC
Train	0.745	0.690	0.628	0.657	0.808
CV (media, 5 folds)	0.684	0.601	0.569	0.584	0.721
Test reservado	0.663	0.581	0.492	0.533	0.693

La brecha entre train y test nos mostró que no debíamos interpretar el desempeño de entrenamiento como una estimación del rendimiento futuro. Usamos la SVM como referencia clásica principal y, más adelante, añadimos una comparación RBF sobre las mismas submuestras que utilizamos para la QSVM.

Mostramos las matrices de confusión de train y test en la Figura 2.

**Figura 2:** Matrices de confusión de la línea base SVM–RBF.

3. Kernel cuántico e implementación

Exploramos un análogo cuántico del SVM con el objetivo de estudiar una medida de similitud diferente a la inducida por los kernels clásicos. Mediante un *feature map* cuántico, cada registro $\mathbf{x}$ se codificó en un estado $|\phi(\mathbf{x})\rangle$, y el kernel se obtuvo a partir del solapamiento entre pares de estados. Esta construcción permite representar los datos en un espacio de características definido por el circuito y explorar correlaciones que pueden resultar difíciles de modelar mediante una elección clásica fija. No asumimos, sin embargo, que esta mayor flexibilidad implique automáticamente un mejor desempeño ni una ventaja computacional. Nuestro propósito fue evaluar empíricamente si la geometría inducida por los circuitos cuánticos ofrecía una separación útil para este problema y analizar la viabilidad de su implementación. La construcción de la matriz kernel requirió evaluar las similitudes entre pares de registros, por lo que el número de comparaciones creció como $O(N^2)$. En consecuencia, además de las métricas predictivas, examinamos la profundidad de los circuitos, el número de puertas entrelazantes y el costo de ejecución como factores centrales para determinar la escalabilidad del enfoque.

3.1. Submuestras balanceadas y sin imputación

Seleccionamos las submuestras de manera jerárquica, anidada y reproducible: extrajimos las filas de train únicamente de la partición de entrenamiento original y las filas de test únicamente de la partición reservada. Además, elegimos filas completas antes de la imputación; por tanto, no tuvimos que completar valores faltantes en ninguna observación seleccionada. Usamos las mismas cinco variables de la línea base clásica para mantener una comparación directa y asignamos un qubit a cada una.

Tomamos dos muestras de entrenamiento, de 16 y 32 filas, balanceadas para evitar que una clase dominara el experimento. Como construimos el kernel comparando las filas por pares, resumimos las escalas resultantes en el Cuadro 3.

Cuadro 3: Escalas del experimento.

Tamaño	Train (0/1)	Test (0/1)	K_{train}	K_{test}	Circuitos/mapa
16	8/8	4/4	16×16	8×16	396
32	16/16	8/8	32×32	16×32	1,624

3.2. Mapas de características y estimación de fidelidad

Para una observación $\mathbf{x}$, cada mapa preparó $|\phi(\mathbf{x})\rangle = U(\mathbf{x})|0\rangle^{\otimes 5}$. Implementamos los mapas en Pytket y los cargamos en programas Guppy: (i) **ZZ**, con H , $R_z(x_i)$ y correlaciones ZZ entre todos los pares; (ii) **Pauli** $Z + YY$, con términos Z individuales e interacciones YY lineales entre vecinos; y (iii) **Custom**, con $R_y(x_i)$, una cadena lineal de CX y $R_x(x_i)$. Cada mapa usó cinco qubits, uno por variable. Custom fue el mapa menos profundo. Antes de formar $U^\dagger U$, usamos 20 puertas CX por $U(\mathbf{x})$ en ZZ, ocho en Pauli $Z + YY$ y cuatro en Custom.

3.3. Ejecución y costo

Ejecutamos las seis familias (tres mapas por dos tamaños) de forma local en Selene con `StatevectorSimulator`, semilla 42 y 1,000 shots por circuito. Calculamos K_{train} en el triángulo superior, lo reflejamos y fijamos su diagonal en uno. Para K_{test} , apilamos $[X_{\text{test}}, X_{\text{train}}]$, calculamos la matriz conjunta sin diagonal y recortamos el bloque test–train. Este procedimiento fue correcto, pero con él recalculamos bloques que después descartamos: 396 circuitos por mapa para el tamaño 16 y 1,624 para el tamaño 32. En total ejecutamos 6,060 circuitos, equivalentes a 6.06 millones de shots. Observamos un crecimiento cuadrático, coherente con el costo $O(N^2)$ de construir una matriz de Gram. Identificamos como mejora posterior calcular directamente solo los pares test–train.

4. QSVM y resultados

Entrenamos cada modelo con `SVC(kernel="precomputed", class_weight="balanced")`. Para cada familia, seleccionamos $C \in \{10^{-6}, 0.1, 1, 10\}$ según el F1 obtenido en cinco folds estratificados de K_{train} , y mantuvimos K_{test} aislada hasta la evaluación final. Interpretamos la desviación mostrada en el Cuadro 4 como la variabilidad entre folds, no como una barra de error de hardware ni de repeticiones independientes.

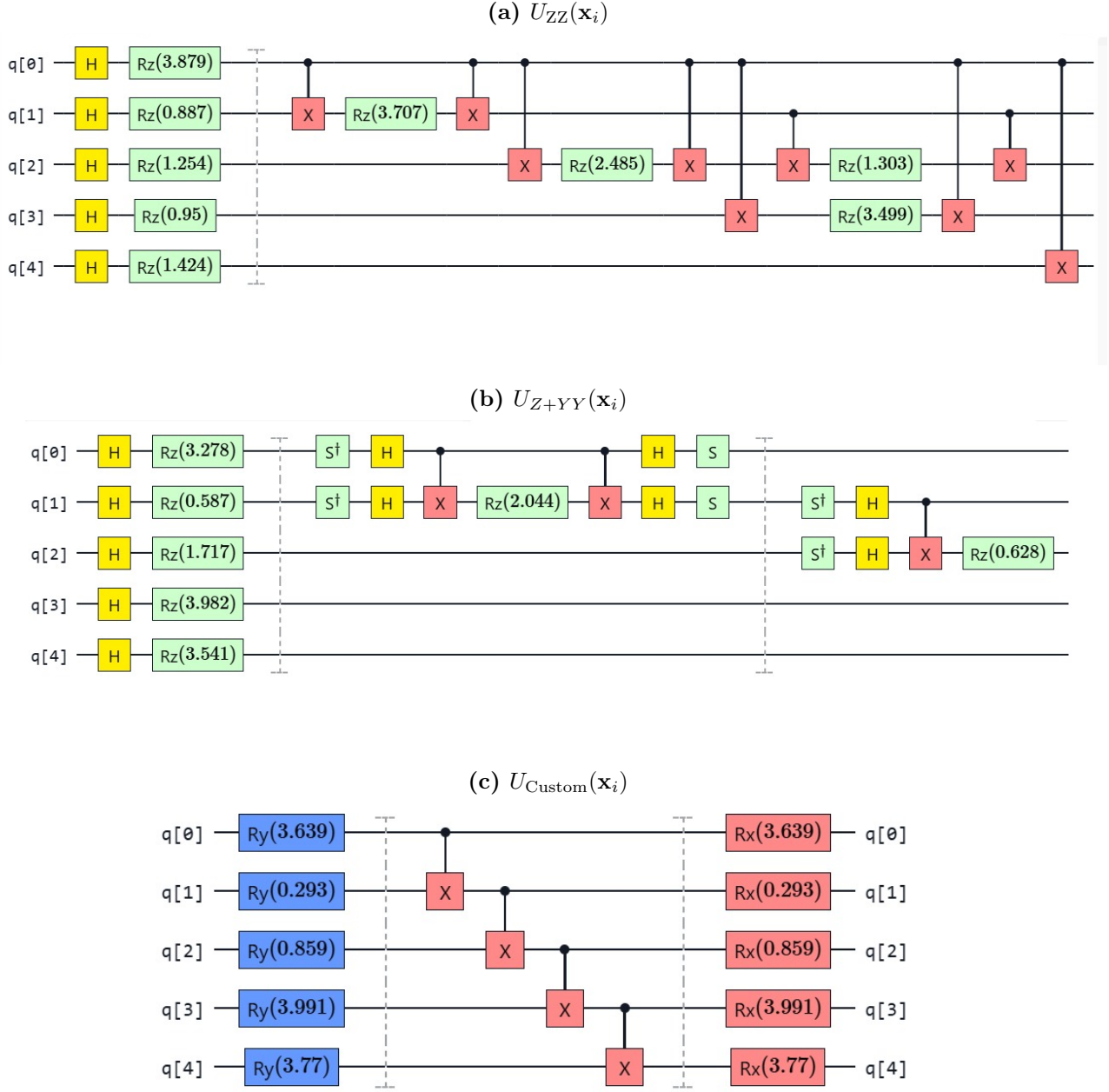
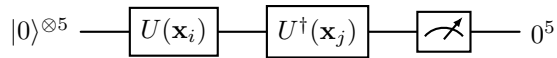


Figura 3: Circuitos asociados con $U(\mathbf{x}_i)$ para los tres mapas de características evaluados: (a) ZZ, (b) Pauli $Z + YY$ y (c) personalizado R_y -CX- R_x . Los valores mostrados correspondieron a observaciones de ejemplo y cambiaron con cada vector de entrada $\mathbf{x}_i$.



$$K_Q(\mathbf{x}_i, \mathbf{x}_j) = |\langle \phi(\mathbf{x}_j) | \phi(\mathbf{x}_i) \rangle|^2 = P(0^5 | U^\dagger(\mathbf{x}_j)U(\mathbf{x}_i)).$$

Figura 4: Circuito de fidelidad común a los tres mapas. De izquierda a derecha, preparamos $U(\mathbf{x}_i)$, aplicamos el inverso $U^\dagger(\mathbf{x}_j)$ y estimamos el kernel mediante la frecuencia del resultado 0^5 .

Cuadro 4: Resultados QSVM. BA: exactitud balanceada; AUC: ROC–AUC.

Mapa	n_{tr}/n_{te}	C	F1 CV (media $\pm$ DE)	BA	Precisión	Recall	F1 test	AUC
Custom	16/8	0.1	0.667 ± 0.365	0.375	0.429	0.750	0.545	0.438
Custom	32/16	1	0.624 ± 0.228	0.375	0.375	0.375	0.375	0.484
ZZ	16/8	10^{-6}	0.300 ± 0.245	0.375	0.333	0.250	0.286	0.313
ZZ	32/16	10	0.381 ± 0.206	0.375	0.400	0.500	0.444	0.375
Pauli Z + YY	16/8	1	0.367 ± 0.306	0.625	0.667	0.500	0.571	0.688
Pauli Z + YY	32/16	1	0.525 ± 0.147	0.625	0.625	0.625	0.625	0.766

4.1. Análisis visual de las matrices kernel

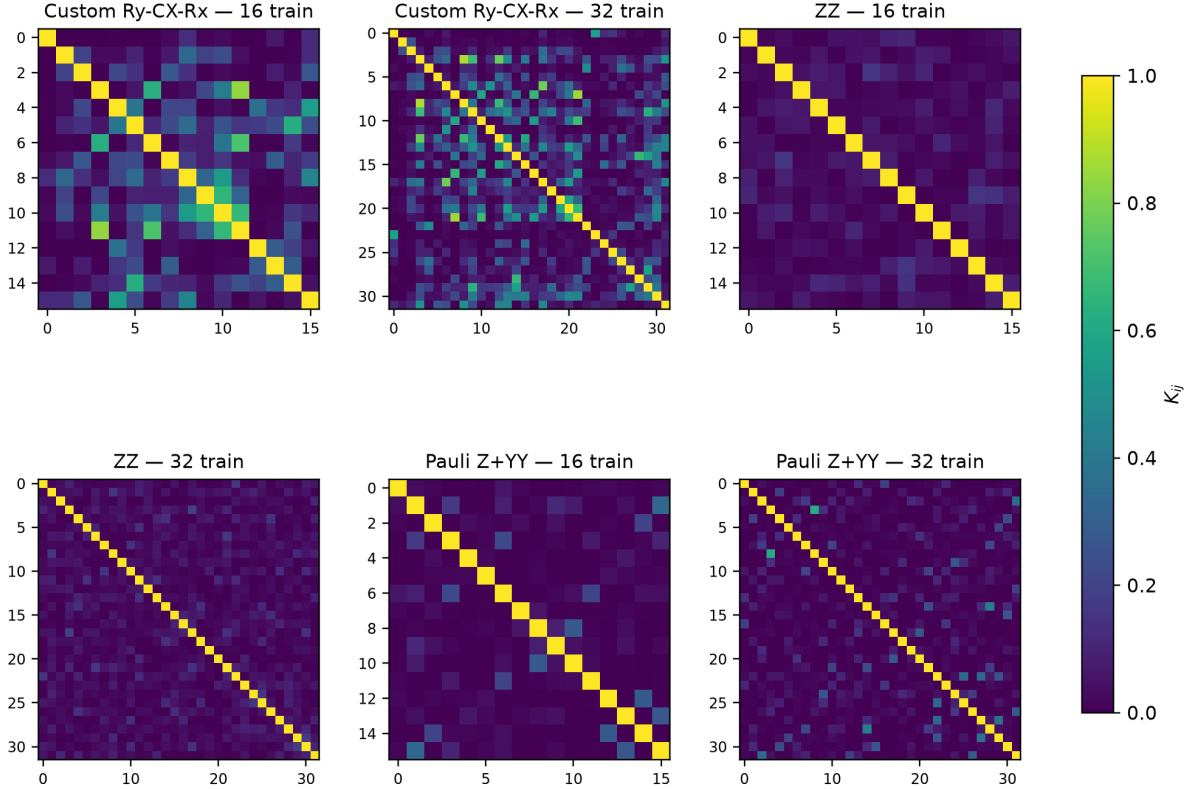


Figura 5: Matrices K_{train} de los mapas Custom, ZZ y Pauli Z + YY con 16 y 32 observaciones de entrenamiento. Todas las matrices se muestran en la misma escala de fidelidad, de 0 (oscuro) a 1 (amarillo).

La Figura 5 mostró tres geometrías claramente diferentes. En **ZZ**, casi toda la intensidad quedó concentrada en la diagonal y las similitudes fuera de ella fueron cercanas a cero. Por tanto, el mapa codificó observaciones distintas como estados casi ortogonales y produjo un kernel próximo a la identidad. Esta separación extrema memorizó la individualidad de cada registro, pero ofreció poca estructura compartida para generalizar por clase; esto fue consistente con su bajo desempeño en test.

Para el mapa **Custom** muchas observaciones quedaron demasiado próximas en el espacio inducido, lo que redujo el contraste entre grupos. Aunque Custom alcanzó una alineación kernel–objetivo de 0.295 con 16 filas, ese patrón no se sostuvo al aumentar a 32 observaciones y su F1 de test disminuyó de 0.545 a 0.375.

Pauli Z + YY presentó el equilibrio más útil entre ambos extremos: mantuvo una diagonal dominante, pero conservó similitudes fuera de ella con mayor estructura que ZZ y sin la saturación observada en Custom. Entre los tres mapas evaluados, esta geometría resultó la más adecuada para los datos: al pasar de 16 a 32 filas mejoró el F1 de test de 0.571 a 0.625 y el ROC–AUC de 0.688 a 0.766. Aun así, la cercanía de la matriz a la identidad y el reducido tamaño de la muestra impidieron interpretar el resultado como una ventaja cuántica concluyente. Todas las matrices conservaron un valor propio mínimo positivo, por lo que no fue necesario proyectarlas al cono semidefinido positivo.

Cuadro 5: Comparación directa en las mismas filas.

Modelo	n_{tr}/n_{te}	Exactitud	Precisión	Recall	F1
SVM-RBF	16/8	0.500	0.500	1.000	0.667
Mejor QSVM (Pauli)	16/8	0.625	0.667	0.500	0.571
SVM-RBF	32/16	0.563	0.545	0.750	0.632
Mejor QSVM (Pauli)	32/16	0.625	0.625	0.625	0.625

Al comparar los modelos sobre las mismas filas, evitamos una comparación injusta entre una QSVM evaluada con 16 casos de test y una SVM clásica evaluada con 656. Con 32 muestras de entrenamiento, Pauli obtuvo mayor exactitud y precisión, mientras que RBF conservó un F1 apenas superior debido a su mayor recall. Como el test solo contenía ocho positivos y ocho negativos, cambiar la clasificación de una observación habría modificado las métricas en varios puntos porcentuales; por ello no encontramos evidencia estadística de superioridad.

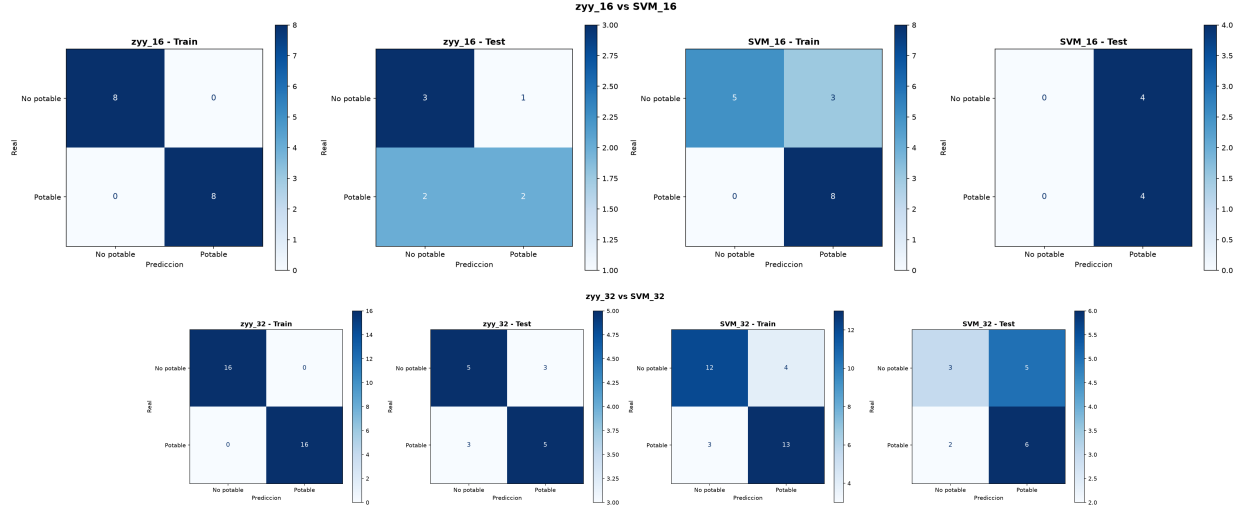


Figura 6: Comparación de las matrices de confusión de la mejor QSVM y la SVM-RBF sobre las mismas filas, para 16 (arriba) y 32 (abajo) muestras de entrenamiento.

5. Limitaciones, reproducibilidad y trabajo pendiente

- Trabajamos con submuestras deliberadamente pequeñas y usamos una sola selección anidada con semilla 42. Observamos una dispersión CV alta y no realizamos repeticiones completas con distintas semillas de muestreo y de shots.
- Al usar Selene *statevector*, no modelamos el ruido de un dispositivo, aunque el muestreo de 1,000 shots discretizó las fidelidades. No ejecutamos el experimento en una computadora cuántica ni en el emulador H2, por lo que no evaluamos conectividad, compilación física, ruido ni mitigación.
- Seleccionamos previamente las cinco variables con el flujo clásico. Con ello redujimos qubits y costo, pero condicionamos la geometría cuántica al criterio de selección clásico. No exploramos repeticiones, topologías ni reescalados de ángulos.
- No establecimos una ventaja cuántica. Una afirmación de ese tipo requeriría comparaciones de escalabilidad, estimaciones de incertidumbre, líneas base clásicas más fuertes y ejecución en hardware cuántico.

Concentramos el flujo reproducible en `qsvm_water.ipynb` y las funciones reutilizables en `funciones.py`. Guardamos las matrices finales en `data/runs` con la nomenclatura `{ZZ|zyy|Custom}_{Train|Test}_{16|32}.csv`, y las figuras en `images`. El entorno se puede reconstruir a partir de `requirements.txt`.

Declaración sobre lenguaje y SDK : Elegimos Python porque nos permitió integrar el preprocesamiento de pandas/scikit-learn con Pytket, Guppy y Selene en un flujo verificable. Con Pytket expresamos los tres mapas puerta por puerta y reutilizamos el mismo constructor $U^\dagger(\mathbf{x}_j)U(\mathbf{x}_i)$; con Guppy compilamos y ejecutamos el programa del kernel, y con Selene realizamos pruebas locales reproducibles con rotaciones arbitrarias. Generamos correctamente K_{train} y K_{test} , guardamos las matrices y las usamos con `SVC(kernel="precomputed")`. Encontramos como principales fricciones el costo cuadrático y el cálculo de bloques de la matriz conjunta de test que luego descartamos. No completamos la ejecución en un backend cuántico o H2, no registramos la profundidad ni las puertas después de la transpilación para ese dispositivo y tampoco repetimos las matrices con varias semillas para cuantificar la incertidumbre asociada a los shots y al ruido.

Referencias

- [1] Quantathon CR 2026. *Challenge 2: Hacia el agua limpia para todos*. Documento de especificación y rúbrica, 2026.
- [2] A. Kadiwal. *Water Potability*. Kaggle, consultado en 2026. <https://www.kaggle.com/datasets/adityakadiwal/water-potability>.
- [3] C. Cortes y V. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995. <https://doi.org/10.1007/BF00994018>.
- [4] M. Schuld y N. Killoran. Quantum machine learning in feature Hilbert spaces. *Physical Review Letters*, 122:040504, 2019. <https://doi.org/10.1103/PhysRevLett.122.040504>.
- [5] V. Havlíček et al. Supervised learning with quantum-enhanced feature spaces. *Nature*, 567:209–212, 2019. <https://doi.org/10.1038/s41586-019-0980-2>.
- [6] Naciones Unidas. *Objetivo 6: Agua limpia y saneamiento*. Agenda 2030 para el Desarrollo Sostenible. <https://sdgs.un.org/goals/goal6>.